Lesson 08 · ASP.NET Core Web APIs

Hosting, dependency injection, minimal APIs, OpenAPI, resilient partner calls and background work in ASP.NET Core 10 — mapped from Spring Boot, NestJS and Express

Contents

1 · Why this lesson — the service you have built many times, rebuilt in .NET 2
- 1.1 · The numbers to leave this lesson with 2

2 · Hosting, configuration and options 2
- 2.1 · The request pipeline of L08.QuoteApi 3
- 2.2 · Program.cs, part 1 — the services 3
- 2.3 · Program.cs, part 2 — the pipeline, next to Express 4
- 2.4 · Concept map — Spring Boot, NestJS and Express → ASP.NET Core 4
- 2.5 · Configuration layers and options validation, run for real 5
- 2.6 · A feature registration in C# and in Visual Basic 5

3 · Dependency injection — lifetimes are a design decision 6
- 3.1 · Two requests, three lifetimes 6
- 3.2 · Choosing a lifetime 6
- 3.3 · A captive dependency, unvalidated and validated 7
- 3.4 · Registrations by lifetime in the API 7

4 · Endpoints — minimal APIs, typed results, filters and validation 7
- 4.1 · Routes — a Spring controller vs a minimal API route group 8
- 4.2 · Handlers that return typed results 8
- 4.3 · Request contracts validated by AddValidation 8
- 4.4 · An endpoint filter for a rule that needs a service 9
- 4.5 · The controller you know — an [ApiController] 9
- 4.6 · Minimal API or controller? 10
- 4.7 · A minimal API endpoint written in Visual Basic 10
- 4.8 · POST /quotes through middleware, filters and the VB calculator 10

5 · OpenAPI and ProblemDetails — a contract that tells the truth 11
- 5.1 · Status codes per operation — documented vs received in the tour 11
- 5.2 · Who writes each error response in the sample 11
- 5.3 · An IExceptionHandler that maps a partner outage to 503 12

6 · Cross-cutting middleware — decide the defaults 12
- 6.1 · Three features whose defaults need a decision 13
- 6.2 · A rate-limit policy that reads validated options 13

7 · Calling ~30 rating partners — HttpClientFactory and resilience 13
- 7.1 · The standard resilience handler, outermost strategy first 14
- 7.2 · Registering the typed client and its pipeline 14
- 7.3 · The typed client — retries happen below this code 14
- 7.4 · Circuit breaker states with the standard defaults 15
- 7.5 · Configuring the handler — no retried POST, an early circuit breaker 15
- 7.6 · Partner attempts per scenario in L08.PartnerLab 16
- 7.7 · What the partner lab prints 16

8 · Background work — hosted services 16
- 8.1 · Accepting a quote returns before the notification is sent 17
- 8.2 · The notification dispatcher 17
- 8.3 · Keyed channels and the hosted service 18

9 · Hands-on — run the API in memory and test it the same way 18
- 9.1 · Commands 18
- 9.2 · What you should see 18
- 9.3 · The test factory — the real Program with two seams 19
- 9.4 · Lesson 08 tests by area 19
- 9.5 · Web-tagged SDK templates by language 19

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why this lesson — the service you have built many times, rebuilt in .NET

You have shipped this service already: an HTTP API with validation, configuration per environment, a DI container, calls to unreliable partners and a background worker. You built it in Spring Boot and WebFlux, in NestJS and Express, and you wired a motor-insurance platform to ~30 external rating partners. ASP.NET Core does the same jobs; this lesson spends its words on where the .NET shape differs and where its defaults would surprise you.

The biggest shift is from discovery to declaration. Spring finds beans by scanning and configures itself from the classpath. An ASP.NET Core service states everything in `Program.cs`: each service and its lifetime, each middleware in the order it runs, each endpoint. That file is the architecture diagram, so reviewing it is where you catch design problems.

The running example is the MotorQuote API. `L08.QuoteApi` creates, reads and accepts quotes, prices them with rating rules written in Visual Basic (`L08.Rating.Vb`), calls rating partners through a resilient typed client, and notifies the customer from a background service. It prices with the track's canonical tariff — illustrative rates, not a real one — and its quote store stays in memory for the whole lesson: persistence is not this lesson's subject. lesson 09 (Data Access with EF Core) models the same quotes in a real database, a `DbContext` and migrations in projects of its own rather than by editing this API; lesson 11 (Security & Identity with Entra ID) secures an API like this one.

1.1 · The numbers to leave this lesson with

OPENAPI VERSION EMITTED	RETRIES, STANDARD HANDLER	RATE-LIMIT REJECTION	LESSON 08 TESTS	SAMPLE CODE
3.1	**3**	**503**	**32**	**1542 lines**
.NET 10 default · verified · sample serves 3.1.1	exponential + jitter, all methods · verified	default status, not 429 · verified	in-memory API tests · measured	C# + VB, 6 projects · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured architecture

i Before you start

1. **You need records and interfaces from lesson 03 (Types, OOP & Generics); `async/await`, cancellation and channels from lesson 05 (Async, Tasks & Concurrency); and project references from lesson 07 (Solutions, MSBuild & Mono-repos).** DI, SOLID and HTTP design are assumed; the lesson explains their .NET form, not the ideas.
2. **You will build and run 6 projects:** the `L08.QuoteApi` web API, the `L08.Rating.Vb` Visual Basic library it calls, `L08.QuoteApi.Tests` (32 in-memory tests), and three console apps — `L08.HostingLab` (configuration and DI), `L08.PartnerLab` (resilience) and `L08.ApiTour` (the API as a client sees it). None opens a port or touches the network.
3. **Chips:** • Same idea transfers as-is, • Renamed is the same idea under another name, • Different behaves differently, • Trap looks the same and bites. Facts that change between releases are • Verified with a link; numbers from this repository are • Measured; judgements are • Estimate. Kotlin and TypeScript panels are for comparison and are not compiled.

2 · Hosting, configuration and options

An ASP.NET Core service is a console program that builds a host, and what Spring Boot does by scanning happens in explicit lines of `Program.cs`. `WebApplication.CreateBuilder(args)` sets up configuration, logging, the DI container and Kestrel, the in-process web server that plays embedded Tomcat's or Netty's part. You register services on `builder.Services`, call `Build()`, add middleware and endpoints to `app`, and call `Run()`.

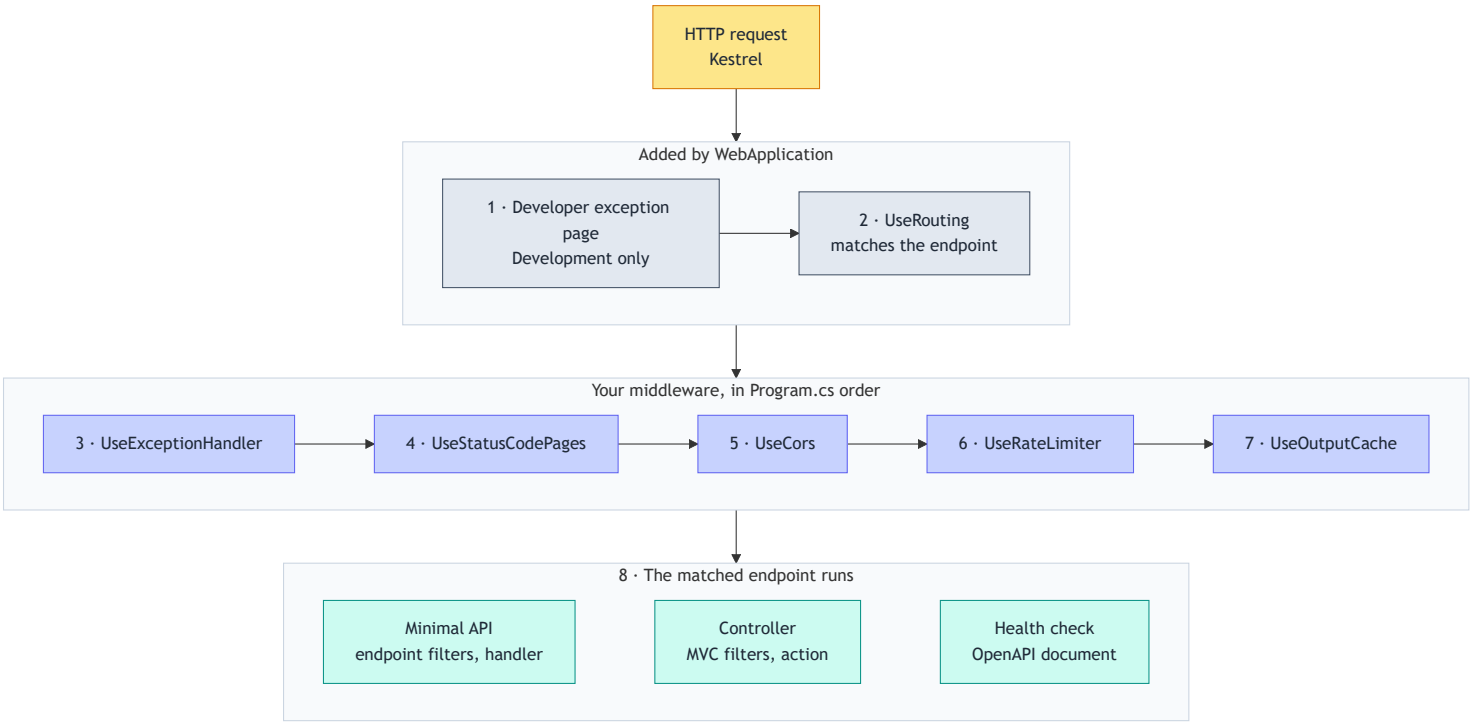
Middleware runs in the order of the lines — after one step you did not write. As in Express, each `Use…` wraps everything after it. When you map endpoints, `WebApplication` adds `UseRouting` itself, ahead of your middleware (WebApplication and WebApplicationBuilder), so the endpoint is already chosen when `UseCors` and `UseRateLimiter` run. Unlike Express, the exception handler goes first: it can only catch what runs inside it.

Configuration is a stack of sources where the last one wins, and the options pattern is your `@ConfigurationProperties`. For a web app, from highest priority: command line, environment variables, user secrets (Development only), `appsettings.{Environment}.json`, `appsettings.json` (Configuration in ASP.NET Core). An environment variable spells `Quotes:ValidityDays` as `Quotes__ValidityDays`. Bind a section to a class, validate it with DataAnnotations and call `ValidateOnStart()` — without that call a bad value fails the first request that reads it instead of failing the deployment.

Logging is injected like any service: ask for `ILogger<T>` and you get the SLF4J logger for class `T`. The category is `T`'s full name, and `Logging:LogLevel` in `appsettings.json` sets the minimum level per category prefix — the sample raises `Microsoft.AspNetCore` to Warning. A message is a template: in `"Partner {PartnerId} unavailable"` the placeholder is

filled by position, like SLF4J's {}, and its name becomes a property that a structured log store can query (Logging in .NET). Build the message with $"…" and that property is gone. Log sinks, OpenTelemetry and CloudWatch belong to lesson 12 (Cloud-Native .NET on AWS & Modernization).

2.1 · The request pipeline of L08.QuoteApi



numbers = run order · amber = the incoming request · slate = added by WebApplication · indigo = middleware in Program.cs order · teal = the endpoint that runs · a response travels back through the same steps in reverse

2.2 · Program.cs, part 1 — the services

C# — samples/L08.QuoteApi/Program.cs · #region services

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<PartnerExceptionHandler>();
builder.Services.AddValidation();                          // new in .NET 10
builder.Services.AddOpenApi();
builder.Services.AddHealthChecks().AddCheck<PartnerHealthCheck>("partners");
builder.Services.ConfigureHttpJsonOptions(o =>             // minimal APIs
    o.SerializerOptions.Converters.Add(new JsonStringEnumConverter()));
builder.Services.AddControllers().AddJsonOptions(o =>      // controllers: separate options
    o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));

builder.Services.AddQuoteFeature(builder.Configuration); // C# registration
builder.Services.AddMotorRating(builder.Configuration);  // the same, written in VB
builder.Services.AddPartnerClients(builder.Configuration);
builder.Services.AddNotifications();
builder.Services.AddQuoteRateLimits();
builder.Services.AddOutputCache();
builder.Services.AddCors(cors => cors.AddPolicy("broker-portal",
    policy => policy.WithOrigins("https://brokers.example").AllowAnyHeader()));
```

Read it as the Spring application class with the scanning written out. Framework features first — ProblemDetails, the exception handler, .NET 10 validation, OpenAPI, health checks — then one `Add…` call per feature registration, C# and VB side by side, then the cross-cutting policies of §6. Look twice at the JSON lines: minimal APIs read `ConfigureHttpJsonOptions` and controllers read `AddJsonOptions`, two separate option sets. Configure one and the other endpoint style keeps its defaults — the test `Minimal_APIs_and_controllers_read_separate_json_options` clears the MVC set and only the controller's enum turns into a number.

2.3 · Program.cs, part 2 — the pipeline, next to Express

TypeScript — for comparison — an Express service you know

```ts
const app = express();

// Express runs middleware in registration order too
app.use(express.json());
app.use(cors({ origin: "https://brokers.example" }));
app.use("/quotes", rateLimit({ windowMs: 60000, limit: 20 }));

app.get("/openapi.json", (_req, res) => res.json(spec));
app.get("/health", healthHandler);
app.use("/partners", partnersRouter);
app.use("/quotes", quotesRouter);

// the error handler is registered LAST in Express
app.use((err, _req, res, _next) =>
  res.status(500)
     .type("application/problem+json")
     .json({ title: "Unexpected error", status: 500 }));

app.listen(5174);
```

C# — L08.QuoteApi/Program.cs · #region pipeline

```cs
var app = builder.Build();

app.UseExceptionHandler();     // exceptions -> ProblemDetails
app.UseStatusCodePages();      // bodiless 4xx/5xx too
app.UseCors("broker-portal");  // after the implicit routing
app.UseRateLimiter();
app.UseOutputCache();

if (!app.Environment.IsProduction())
    app.MapOpenApi();          // GET /openapi/v1.json
app.MapHealthChecks("/health");
app.MapControllers();          // PartnersController
app.MapQuotes();               // minimal API group, C#
app.MapTariff().CacheOutput(); // minimal API group, VB

app.Run();
```

> **Same idea, two differences.** Express registers its error middleware last because errors are passed forward to it; `UseExceptionHandler` goes first because it catches what runs inside it. And there is no `UseRouting` line: `WebApplication` inserts it before the first middleware. The OpenAPI document is mapped everywhere except Production — the web API template maps it in Development only (OpenAPI support in ASP.NET Core).

2.4 · Concept map — Spring Boot, NestJS and Express → ASP.NET Core

You already know	In .NET	Kind	Watch for
`SpringApplication.run` · `NestFactory.create`	`WebApplication.CreateBuilder(args)`	• Renamed	Register every service before `Build()`; the container is fixed after it
Embedded Tomcat / Netty · Node `http`	Kestrel, in process	• Same idea	Behind a load balancer or reverse proxy it is still Kestrel; under IIS the default in-process model uses IIS HTTP Server instead (In-process hosting with IIS) — lesson 12 (Cloud-Native .NET on AWS & Modernization)
Servlet filters · Express `app.use`	Middleware: `app.UseX()`	• Same idea	Line order is run order; routing is inserted before your first middleware
Component scanning + `@Autowired`	Explicit `builder.Services.Add…` calls	• Different	Nothing is discovered; an unregistered type fails when it is resolved
`@ConfigurationProperties` + `@Validated`	Options pattern: `AddOptions<T>().Bind(…)`	• Renamed	Add `ValidateOnStart()`, or bad config fails on first use
`application-{profile}.yml` · `NODE_ENV`	`appsettings.{Environment}.json`	• Renamed	Picked by `ASPNETCORE_ENVIRONMENT`; variables use `__` for `:`
SLF4J `Logger` · `@Slf4j` · NestJS `Logger`	`ILogger<T>` + message templates	• Trap	Placeholders are named properties; a `$"…"` message loses them
Jackson `ObjectMapper` configuration	`ConfigureHttpJsonOptions` · `AddJsonOptions`	• Trap	Minimal APIs and controllers read separate JSON options — set both
Spring request scope · NestJS `Scope.REQUEST`	`AddScoped`	• Trap	A singleton holding it is caught only in Development — §3
`@Qualifier("line")`	Keyed services: `[FromKeyedServices("line")]`	• Renamed	An unkeyed `GetService<T>()` does not see keyed registrations
`@RestController` · NestJS `@Controller`	Minimal API `MapPost`, or `ControllerBase`	• Different	Minimal APIs are the recommended default for new projects — §4
`HandlerInterceptor` · NestJS interceptors and pipes	Endpoint filters (`IEndpointFilter`)	• Renamed	Filters see the bound arguments; middleware sees only the raw request
Bean Validation `@Valid` · `class-validator`	DataAnnotations + `AddValidation()` (.NET 10)	• Renamed	Parameters are checked on every endpoint; request types only if declared in the calling assembly

You already know	In .NET	Kind	Watch for
springdoc-openapi · @nestjs/swagger	`AddOpenApi` + `MapOpenApi`	• Renamed	OpenAPI 3.1, no UI in the box; documents only declared status codes
`@ControllerAdvice` · NestJS exception filters	`IExceptionHandler` + ProblemDetails	• Renamed	.NET 10 no longer logs exceptions a handler marks as handled
`WebClient` + Resilience4j	Typed client + standard resilience handler	• Trap	Retries every HTTP method by default — POST included
`@Scheduled` worker · a BullMQ consumer	`BackgroundService`	• Different	Singleton: open a scope per unit of work; an escaping exception stops the host
`@SpringBootTest` + MockMvc · supertest	`WebApplicationFactory<Program>`	• Same idea	Runs the real pipeline in memory; environment defaults to Development

2.5 · Configuration layers and options validation, run for real

C# L08.HostingLab/Program.cs · #region config-precedence

```
Environment.SetEnvironmentVariable(
    "L08LAB_Quotes__ValidityDays", "14");   // __ means :

var config = new ConfigurationBuilder()
    // stand-ins for appsettings.json and .Production.json
    .AddInMemoryCollection([new("Quotes:ValidityDays", "30")])
    .AddInMemoryCollection([new("Quotes:ValidityDays", "21")])
    .AddEnvironmentVariables(prefix: "L08LAB_")
    .AddCommandLine(["--Quotes:ValidityDays=7"])
    .Build();

string[] layers =
[
    "appsettings.json", "appsettings.Production.json",
    "environment", "command line",
];
foreach (var (source, layer) in config.Providers.Zip(layers))
{
    source.TryGet("Quotes:ValidityDays", out var value);
    Console.WriteLine($"   {layer,-28} {value}");
}
var wins = config["Quotes:ValidityDays"];
Console.WriteLine($"   => Quotes:ValidityDays = {wins}");
```

Output — L08.HostingLab, parts 1 and 2 captured run

```
1 configuration: the last source that sets a key wins
   appsettings.json             30
   appsettings.Production.json  21
   environment                  14
   command line                 7
   => Quotes:ValidityDays = 7
2 options validation: fail at start-up, not on first use
   OptionsValidationException
               DataAnnotation validation failed for
               'QuoteOptions' members: 'ValidityDays' with
               the error: 'The field ValidityDays must be
               between 1 and 90.'.
```

Every provider keeps its own value; the last one added answers. The lab stacks four sources the way `CreateBuilder` does and asks each for `Quotes:ValidityDays`; the command line wins with 7. Part 2 binds `ValidityDays = 0` to options with `[Range(1, 90)]` and `ValidateOnStart()`: `StartAsync` throws `OptionsValidationException` before any request is served.

2.6 · A feature registration in C# and in Visual Basic

C# QuoteRegistration.cs · #region register

```
public static IServiceCollection AddQuoteFeature(
    this IServiceCollection services,
    IConfiguration config)
{
    services.AddOptions<QuoteOptions>()
        .Bind(config.GetSection(QuoteOptions.SectionName))
        .ValidateDataAnnotations()
        .ValidateOnStart();

    services.AddSingleton(TimeProvider.System);
    services.AddSingleton<IQuoteStore, InMemoryQuoteStore>();
    services.AddSingleton<PolicyNumbers>();
    services.AddScoped<QuoteService>();
    return services;
}
```

Visual Basic L08.Rating.Vb/RatingModule.vb · #region register

```
<Extension>
Public Function AddMotorRating(
        services As IServiceCollection,
        config As IConfiguration) As IServiceCollection
    services.AddOptions(Of RatingOptions)() _
        .Bind(config.GetSection(RatingOptions.SectionName)) _
        .ValidateDataAnnotations() _
        .ValidateOnStart()

    services.AddSingleton(Of PremiumCalculator)()
    Return services
End Function
```

The same registration in both languages. Each registration binds its own configuration section, validates it at start-up and registers its services; `Program.cs` calls `AddQuoteFeature` and `AddMotorRating` side by side. Both are feature seams inside an assembly, not the assembly-sized module lesson 07 (Solutions, MSBuild & Mono-repos) maps onto a Gradle module. The VB version needs `<Extension>` inside a `Module` — there the VB keyword for a type that holds only shared members — where C# writes `this` in a static class, and `_` to continue a line that ends before the dot. The `RatingOptions` it binds hold the track's canonical rates (§9.2).

3 · Dependency injection — lifetimes are a design decision

The container does what Spring's does, with one change that shows up in code review: every registration names its lifetime. Spring and NestJS default to singletons; here `AddSingleton`, `AddScoped` and `AddTransient` are separate calls. Scoped means one instance per HTTP request, because ASP.NET Core opens a scope for each request and disposes it at the end.

The bug to design against is the captive dependency: a singleton that receives a scoped service keeps the first instance for the life of the process. The container checks for it — scoped services resolved from the root provider or injected into singletons — only when the app runs in Development (Dependency injection: scope validation). In Staging or Production the same code starts and silently shares one instance: in 3.3 two scopes both see `ScopedOp #3`. The sample's test factory turns the checks on for its Testing environment.

Keyed services (.NET 8) replace `@Qualifier`. `AddKeyedScoped<INotificationChannel, LineChannel>("line")` registers one implementation per key; `[FromKeyedServices("line")]` or `GetRequiredKeyedService` resolves it. A plain `GetService<INotifier>()` returns null, because keyed registrations are invisible to unkeyed resolution.

3.1 · Two requests, three lifetimes

C# — L08.HostingLab/Program.cs · #region lifetimes

```
var services = new ServiceCollection()
    .AddTransient<TransientOp>()
    .AddScoped<ScopedOp>()
    .AddSingleton<SingletonOp>();

using (var root = services.BuildServiceProvider())
{
    foreach (var request in new[] { "A", "B" })
    {
        // ASP.NET Core opens one scope per HTTP request
        using var scope = root.CreateScope();
        var sp = scope.ServiceProvider;
        Console.WriteLine($"   request {request}: "
            + $"transient {Ids<TransientOp>(sp)}  "
            + $"scoped {Ids<ScopedOp>(sp)}  "
            + $"singleton {Ids<SingletonOp>(sp)}");
    }
}
```

Output — L08.HostingLab, parts 3 to 5 — captured run

```
3 lifetimes: two requests, each resolving every service twice
   request A: transient 1,2  scoped 1,1  singleton 1,1
   request B: transient 3,4  scoped 2,2  singleton 1,1
4 captive dependency: a singleton holding a scoped service
   unvalidated: scope 1 sees ScopedOp #3, scope 2 sees
                ScopedOp #3
   validated:   Cannot consume scoped service 'ScopedOp' from
                singleton 'PriceCache'.
5 keyed services: one interface, a key per implementation
   key "line" resolves LineNotifier
   unkeyed INotifier    null
```

Read the numbers as instance ids. Each request resolves every service twice: transient gives four instances (1–4), scoped one per request (1, 2), singleton one in total. Part 4 of the output is the captive dependency in 3.3; part 5 is the keyed lookup from the story — the keyed registrations of the API itself are in 8.3.

3.2 · Choosing a lifetime

Lifetime	One instance per	Spring · NestJS	In L08.QuoteApi	Watch for
Singleton	container (process)	singleton (the default) · default scope	`IQuoteStore`, `PolicyNumbers`, `NotificationQueue`, VB `PremiumCalculator`	Shared by concurrent requests: use `ConcurrentDictionary`, `Interlocked`, `Lock`
Scoped	HTTP request (or manual scope)	request scope · `Scope.REQUEST`	`QuoteService`, keyed notification channels	Never inject into a singleton or a hosted service — create a scope
Transient	resolution	prototype · `Scope.TRANSIENT`	`PartnerRateClient` (typed `HttpClient`)	A typed client captured by a singleton stops seeing DNS changes (Use the IHttpClientFactory)

3.3 · A captive dependency, unvalidated and validated

C# — samples/L08.HostingLab/Program.cs · #region captive

```
var captive = new ServiceCollection()
    .AddScoped<ScopedOp>()
    .AddSingleton<PriceCache>();                  // PriceCache(ScopedOp scoped)

using (var lax = captive.BuildServiceProvider())  // no validation: default outside Development
{
    using var first = lax.CreateScope();
    using var second = lax.CreateScope();
    var a = first.ServiceProvider.GetRequiredService<PriceCache>().Scoped.Id;
    var b = second.ServiceProvider.GetRequiredService<PriceCache>().Scoped.Id;
    Say("unvalidated:", $"scope 1 sees ScopedOp #{a}, scope 2 sees ScopedOp #{b}");
}

try
{
    captive.BuildServiceProvider(new ServiceProviderOptions
    {
        ValidateScopes = true,                    // both are switched on in Development
        ValidateOnBuild = true,
    });
}
catch (AggregateException ex)
{
    var cause = ex.InnerExceptions[0];
    Say("validated:", cause.InnerException?.Message ?? cause.Message);
}
```

The bug compiles, starts and passes a happy-path test. `PriceCache` is a singleton whose constructor takes the scoped `ScopedOp`. Built without validation — the default outside Development — the provider hands both scopes the one instance it created first, #3 because part 3 already made #1 and #2. Built with `ValidateScopes` and `ValidateOnBuild`, it throws before any request: *Cannot consume scoped service 'ScopedOp' from singleton 'PriceCache'*. Say prints a label and wraps the text at 62 columns.

3.4 · Registrations by lifetime in the API



AddSingleton / AddScoped / AddTransient calls (keyed or not), typed AddHttpClient and AddHostedService calls in L08.QuoteApi and the VB module · measured at build time

Most services are singletons because the state lives in thread-safe stores. The one request-bound unit is `QuoteService` plus the keyed channels; the typed partner client is transient; the hosted dispatcher is a singleton too, which is why it opens a scope per message (§8). Counted from the sample source at build time; framework calls such as `AddOpenApi` are not counted. • Measured

4 · Endpoints — minimal APIs, typed results, filters and validation

Microsoft recommends minimal APIs for new projects; controllers remain for teams that need MVC's extension points (APIs overview). A minimal API maps a route to a lambda or a static method with `MapGet`/`MapPost`, grouped under a prefix with `MapGroup`. A controller is the `@RestController` you know: an `[ApiController]` class mapped by `MapControllers()`. Both run in one app — this sample has both.

Return typed results, because the return type is the contract. `Results<Ok<PolicyResponse>, NotFound, Conflict<ProblemDetails>>` is checked by the compiler, read by the OpenAPI generator (§5) and asserted by unit tests without HTTP. Endpoint filters are the `HandlerInterceptor` of minimal APIs: they wrap the handler and see its bound arguments.

.NET 10 validates minimal API arguments from DataAnnotations after one call, `AddValidation()`. A source generator finds the types your endpoints bind — records and nested types included — and an endpoint filter answers 400 with a ValidationProblem. The generator only discovers types in the assembly where `AddValidation` is called (What's new in ASP.NET

Core in .NET 10); 4.7 shows what that means for a Visual Basic library. A rule that needs a service — today's date from `TimeProvider` — stays in your own filter.

4.1 · Routes — a Spring controller vs a minimal API route group

Kotlin — for comparison Spring Boot

```kotlin
@RestController
@RequestMapping("/quotes")
class QuoteController(private val service: QuoteService) {

    @PostMapping
    @RateLimiter(name = "quote-writes")
    fun create(@Valid @RequestBody request: QuoteRequest)
        : ResponseEntity<QuoteResponse> {
        val quote = service.create(request)
        return ResponseEntity
            .created(URI("/quotes/${quote.quoteId}"))
            .body(quote)
    }

    @GetMapping("/{id}")
    fun get(@PathVariable id: UUID) =
        service.find(id)?.let { ResponseEntity.ok(it) }
            ?: ResponseEntity.notFound().build()

    @PostMapping("/{id}/accept")
    fun accept(@PathVariable id: UUID) = service.accept(id)
}
```

C# QuoteEndpoints.cs · #region map-quotes

```csharp
public static RouteGroupBuilder MapQuotes(
    this IEndpointRouteBuilder app)
{
    var quotes = app.MapGroup("/quotes").WithTags("Quotes");

    quotes.MapPost("/", Create)
        .AddEndpointFilter<StartDateFilter>()
        .RequireRateLimiting(RateLimits.QuoteWrites)
        .ProducesValidationProblem()        // 400
        .ProducesProblem(Status429TooManyRequests);

    quotes.MapGet("/{id:guid}", Get);
    quotes.MapPost("/{id:guid}/accept", Accept);
    return quotes;
}
```

Annotations become method calls on the route. `@RequestMapping` is `MapGroup`, `@PostMapping` is `MapPost`, `@RateLimiter` is `RequireRateLimiting`. The `:guid` constraint makes a malformed id a 404 before the handler runs. The two `Produces…` calls document responses that filters and middleware produce — §5 shows why they are needed. The handlers follow in 4.2.

4.2 · Handlers that return typed results

C# samples/L08.QuoteApi/Quotes/QuoteEndpoints.cs · #region handlers

```csharp
private static Created<QuoteResponse> Create(QuoteRequest request, QuoteService service)
{
    var quote = service.Create(request);
    return TypedResults.Created($"/quotes/{quote.QuoteId}", quote);
}

private static Results<Ok<QuoteResponse>, NotFound> Get(Guid id, QuoteService service) =>
    service.Find(id) is { } quote ? TypedResults.Ok(quote) : TypedResults.NotFound();

private static Results<Ok<PolicyResponse>, NotFound, Conflict<ProblemDetails>> Accept(
    Guid id, QuoteService service) => service.Accept(id) switch
{
    { Outcome: AcceptOutcome.Accepted, Policy: { } policy } => TypedResults.Ok(policy),
    { Outcome: AcceptOutcome.NotFound } => TypedResults.NotFound(),
    var failed => TypedResults.Conflict(new ProblemDetails
    {
        Title = "Quote cannot be accepted",
        Detail = $"The quote is {failed.Outcome}.",
        Status = StatusCodes.Status409Conflict,
    }),
};
```

Parameters are bound by type and source: `Guid id` from the route, `QuoteRequest` from the JSON body, `QuoteService` from the request scope. `Accept` maps a domain outcome to three HTTP results with a switch expression; its union return type is what makes the OpenAPI document list 200, 404 and 409.

4.3 · Request contracts validated by AddValidation

C# samples/L08.QuoteApi/Quotes/QuoteContracts.cs · #region contracts

```csharp
public sealed record QuoteRequest(
    [Required] VehicleDto Vehicle,
    [Required] DriverDto Driver,
    CoverageClass Coverage,
    DateOnly StartDate,
    [AllowedValues("sms", "email", "line")] string NotifyVia = "email");
```

```
public sealed record VehicleDto(
    [Required, StringLength(40)] string Make,
    [Required, StringLength(40)] string Model,
    [Range(1990, 2035)] int Year,
    [Range(50, 8000)] int EngineCc,
    VehicleUse Use,
    [Range(typeof(decimal), "50000", "20000000")] decimal SumInsured);

public sealed record DriverDto(
    DateOnly DateOfBirth,
    [Range(0, 70)] int LicenceYears,
    [Range(0, 20)] int ClaimsLast5Years);
```

Bean Validation annotations, in attribute form, on positional records. `[AllowedValues]` and `[Range(typeof(decimal), …)]` come from `System.ComponentModel.DataAnnotations`; nested records are validated too, so the tour's invalid body gets 400 with errors for `Vehicle.SumInsured` and `NotifyVia`. These are wire contracts, not the MotorQuote domain types: `SumInsured` is a plain amount because the currency comes from `QuoteOptions`, and responses convert back to `Money`.

4.4 · An endpoint filter for a rule that needs a service

C# — samples/L08.QuoteApi/Quotes/QuoteEndpoints.cs · #region endpoint-filter

```
/// <summary>A rule DataAnnotations cannot express: it needs the clock from DI.</summary>
public sealed class StartDateFilter(TimeProvider clock) : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context, EndpointFilterDelegate next)
    {
        var request = context.GetArgument<QuoteRequest>(0);
        var today = DateOnly.FromDateTime(clock.GetUtcNow().UtcDateTime);

        if (request.StartDate < today || request.StartDate > today.AddDays(60))
        {
            return TypedResults.ValidationProblem(new Dictionary<string, string[]>
            {
                ["StartDate"] = ["Must be between today and 60 days ahead."],
            });
        }
        return await next(context);   // filters run in order before next, reverse after
    }
}
```

A filter is middleware that knows the endpoint's arguments. `GetArgument<QuoteRequest>(0)` reads the bound body; returning a result short-circuits the handler, calling `next` continues. The clock is injected `TimeProvider`, so tests fix the date with `FakeTimeProvider` — lesson 05 (Async, Tasks & Concurrency).

4.5 · The controller you know — an [ApiController]

C# — samples/L08.QuoteApi/Partners/Partners.cs · #region controller

```
[ApiController]
[Route("partners")]
public sealed class PartnersController(
    IPartnerRateClient partners) : ControllerBase
{
    [HttpGet("{partnerId:length(3)}/rates")]
    [ProducesResponseType<PartnerRate>(StatusCodes.Status200OK)]
    public async Task<ActionResult<PartnerRate>> GetRate(
        string partnerId,
        [FromQuery] CoverageClass coverage,
        CancellationToken ct)
    {
        return await partners.GetRateAsync(
            partnerId, coverage, ct);
    }
}
```

Attribute for attribute, this is `@RestController`. `[ApiController]` requires attribute routing, infers binding sources (`partnerId` from the route, a complex type from the body) and answers invalid model state with an automatic 400 ValidationProblem (Create web APIs with ASP.NET Core). `ActionResult<PartnerRate>` returns the value or any result. `[ProducesResponseType]` declares only 200, so the OpenAPI document lists only 200; the 503 that §5's exception handler writes for this route is missing (5.1).

4.6 · Minimal API or controller?

Start with a minimal API when

- The service is new — it is Microsoft's recommended default (APIs overview)
- Endpoints are thin and the logic lives in services, as it should in a microservice
- You want typed results and endpoint filters without the MVC filter stack
- You may publish with Native AOT later — lesson 12 (Cloud-Native .NET on AWS & Modernization)

Keep or choose a controller when

- You need custom model binders (`IModelBinder`) or `IModelValidator`
- You rely on application parts, the application model or OData
- A large existing MVC codebase already has conventions and filters that work
- Its responses are documented by attributes such as `[ProducesResponseType]`, as `PartnersController` in 4.5 is

4.7 · A minimal API endpoint written in Visual Basic

Visual Basic — `samples/L08.Rating.Vb/RatingModule.vb · #region tariff-endpoint`

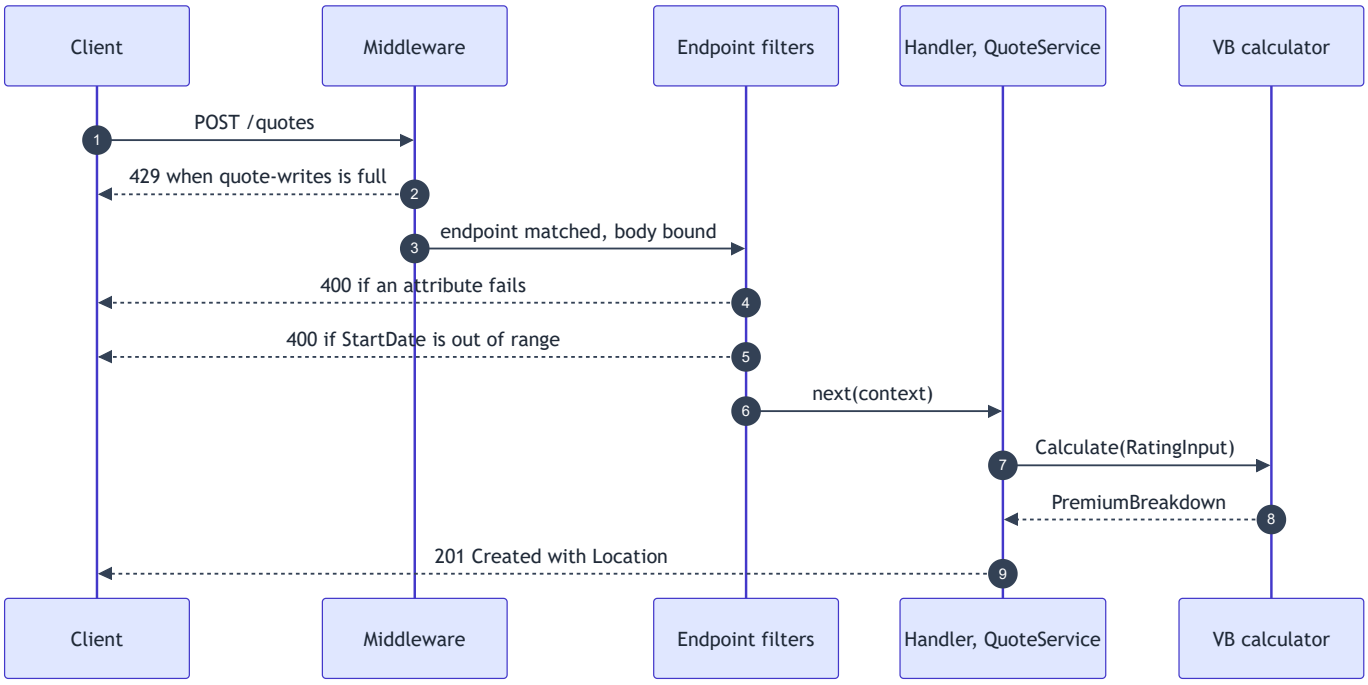
```
' A minimal API endpoint written in VB and mapped into the C# host.
<Extension>
Public Function MapTariff(app As IEndpointRouteBuilder) As RouteGroupBuilder
    Dim group = app.MapGroup("/tariff").WithTags("Tariff")

    group.MapGet("/{coverage}",
        Function(coverage As CoverageClass, calculator As PremiumCalculator)
            Return TypedResults.Ok(New TariffRow(coverage, calculator.BaseRateFor(coverage)))
        End Function)

    Return group
End Function
```

Endpoints are just extension methods on `IEndpointRouteBuilder`, so a VB library can contribute them. The C# host calls `app.MapTariff().CacheOutput()`. The VB lambda binds `coverage` from the route and `PremiumCalculator` from DI exactly as a C# handler would. Validation follows only half-way. `AddValidation`'s filter runs on every endpoint, so DataAnnotations on a VB handler's *parameters* are enforced; but its source generator sees only types declared in the assembly that calls it — the C# host — so attributes on a request *class* written in VB are silently ignored and the request gets 200 (Validation in ASP.NET Core). Declare request DTOs in C#, or validate VB types explicitly. `ValidationScopeTests` pins both behaviours against VB endpoints in `ValidationScope.vb`.

4.8 · POST /quotes through middleware, filters and the VB calculator



solid arrows = calls · dashed arrows = returns · each 400 or 429 short-circuits the rest

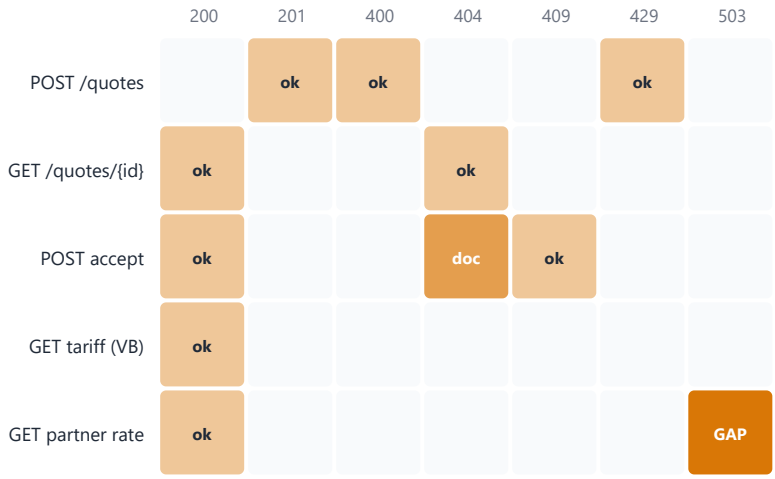
5 · OpenAPI and ProblemDetails — a contract that tells the truth

OpenAPI generation is built in: `AddOpenApi()` and `MapOpenApi()`, and .NET 10 emits OpenAPI 3.1 using Microsoft.OpenApi 2.0 (What's new in ASP.NET Core in .NET 10). Swashbuckle was removed from the web API template in .NET 9 (Swashbuckle removal announcement). The template maps only the JSON document, only in Development (OpenAPI support in ASP.NET Core); a browsing UI is a package you choose, as swagger-ui was with springdoc. The sample's document reports `3.1.1`.

The document lists only the status codes something declared. Typed results declare their own; `Produces…` calls and `[ProducesResponseType]` declare the rest. What middleware, exception handlers and filters return appears nowhere unless you say so — chart 5.1 compares what the tour received with what the document promises.

Errors become RFC 9457 ProblemDetails when three things are registered. `AddProblemDetails()`, `UseExceptionHandler()` and `UseStatusCodePages()` turn unhandled exceptions and empty 4xx/5xx responses into `application/problem+json` (Handle errors in ASP.NET Core APIs). `IExceptionHandler` is your `@ControllerAdvice`: handlers are singletons, called in registration order until one returns true (IExceptionHandler). Since .NET 10 the middleware no longer logs an exception a handler reports as handled, so log it in the handler.

5.1 · Status codes per operation — documented vs received in the tour



ok = in /openapi/v1.json and received · doc = documented, not triggered by the tour · GAP = received but not documented · blank = neither · parsed from the tour output in 9.2, measured on the build machine

A GAP cell is a response a generated client does not expect. *GET partner rate* returned 503, which its entry does not list — the 503 comes from `PartnerExceptionHandler`, which no attribute declares. A doc cell is only a gap in the tour: *POST accept* documents 404, which the tour never triggered. The POST /quotes row is all ok only because 4.1 adds `ProducesValidationProblem()` and `ProducesProblem(429)`. • Measured

5.2 · Who writes each error response in the sample

Status	Produced by	Body	In the OpenAPI document
400	`AddValidation` filter, or `StartDateFilter`	ValidationProblem with `errors` per field	yes — `ProducesValidationProblem()`
404	`TypedResults.NotFound()`, or a malformed `{id:guid}`	empty result → `UseStatusCodePages` writes problem+json	yes — from the union return type
409	`TypedResults.Conflict(new ProblemDetails …)`	the ProblemDetails you built	yes — from the union return type
429	rate limiter middleware, `RejectionStatusCode = 429`	no body → `UseStatusCodePages` writes problem+json (test)	only via `ProducesProblem(429)`
503	`PartnerExceptionHandler`	ProblemDetails + `partnerId`, `Retry-After: 30`	• no — nothing declares it
500	unhandled exception no handler claims	ProblemDetails from `UseExceptionHandler`	• no — nothing declares it

5.3 · An IExceptionHandler that maps a partner outage to 503

C# samples/L08.QuoteApi/CrossCutting/CrossCutting.cs · #region exception-handler

```
public sealed class PartnerExceptionHandler(
    IProblemDetailsService problems,
    ILogger<PartnerExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext http, Exception exception, CancellationToken ct)
    {
        if (exception is not PartnerUnavailableException partner)
            return false;   // not ours: the next handler, then the default 500

        // .NET 10 no longer logs exceptions a handler reports as handled: log it yourself
        logger.LogWarning("Partner {PartnerId} unavailable: {Reason}",
            partner.PartnerId, partner.Message);

        http.Response.StatusCode = StatusCodes.Status503ServiceUnavailable;
        http.Response.Headers.RetryAfter = "30";
        return await problems.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = http,
            Exception = exception,
            ProblemDetails =
            {
                Status = StatusCodes.Status503ServiceUnavailable,
                Title = "Rating partner unavailable",
                Detail = $"Partner {partner.PartnerId} did not answer after retries.",
                Extensions = { ["partnerId"] = partner.PartnerId },
            },
        });
    }
}
```

Return false for exceptions that are not yours. The next handler, then the middleware's default 500 ProblemDetails, takes over. `IProblemDetailsService` writes the body, so the 503 carries the same shape as every other error plus a `partnerId` extension and a `Retry-After` header (asserted in the partner test). The explicit `LogWarning` exists because .NET 10 stopped logging handled exceptions; its template keeps `PartnerId` and `Reason` as properties you can filter on (§2).

6 · Cross-cutting middleware — decide the defaults

What a Spring service gets from Actuator, Resilience4j and the API gateway ships inside ASP.NET Core as a service registration, a middleware and endpoint metadata. `AddRateLimiter` / `UseRateLimiter` / `RequireRateLimiting`; `AddOutputCache` / `UseOutputCache` / `CacheOutput`; `AddHealthChecks` / `MapHealthChecks`. The registration defines policies, the middleware enforces them, the endpoint opts in. CORS works the same way and holds no surprise for a Spring or NestJS engineer: `AddCors` defines the `broker-portal` policy and `UseCors("broker-portal")` answers the preflight for that one origin only (a test proves it).

Three defaults need an explicit decision before production. The rate limiter rejects with 503, which clients and dashboards read as an outage (RejectionStatusCode). A Degraded health check still returns 200, so a load balancer keeps sending traffic (Health checks in ASP.NET Core). The output cache lives in process memory, so two ECS tasks hold two different caches (Output caching middleware).

Order is part of the configuration. `UseOutputCache` goes after `UseCors` and after authentication, or cached responses bypass them; a rate limiter that reads endpoint policies must run after routing (Rate limiting middleware) — `WebApplication`'s implicit `UseRouting` guarantees that unless you move it. Authentication and authorization middleware belong to lesson 11 (Security & Identity with Entra ID).

6.1 · Three features whose defaults need a decision

L08.QuoteApi

1 Rate limiting

architecture · Different

Named policies registered with `AddRateLimiter`, applied per endpoint with `RequireRateLimiting`, enforced by `UseRateLimiter`.

SPRING	Resilience4j `@RateLimiter`, or a quota at the API gateway
DEFAULT	A rejected request gets **503** — the sample sets 429 (RejectionStatusCode)
SAMPLE	`quote-writes`: fixed window per client IP, limit from options
WATCH	Partitions keyed on client input such as IP are a DoS surface (Rate limiting middleware)

2 Output caching

architecture · Different

Stores whole responses on the server and replays them; opt in per endpoint with `CacheOutput()`.

LIKE	Spring `@Cacheable`, but for the HTTP response, not a method result
DEFAULT	GET/HEAD 200 only, no cookies, no authenticated requests, 60 s
STORE	In-process memory: every instance has its own cache — Redis to share it
SAMPLE	The second `GET /tariff/Class3Plus` carries an Age header (test)

3 Health checks

architecture · Different

`AddHealthChecks().AddCheck<T>()` plus `MapHealthChecks("/health")`.

SPRING	Actuator `/actuator/health`: UP or DOWN, no built-in Degraded state
STATUS	Healthy **200** · Degraded **200** · Unhealthy **503** (Health checks in ASP.NET Core)
BODY	Plain text: `Healthy`, `Degraded` or `Unhealthy`
SAMPLE	`PartnerHealthCheck` reports Degraded after a partner call fails

6.2 · A rate-limit policy that reads validated options

C# — samples/L08.QuoteApi/CrossCutting/CrossCutting.cs · #region rate-limits

```
public static IServiceCollection AddQuoteRateLimits(this IServiceCollection services)
{
    services.AddOptions<RateLimitOptions>()
        .BindConfiguration(RateLimitOptions.SectionName)
        .ValidateDataAnnotations()
        .ValidateOnStart();

    return services.AddRateLimiter(limiter =>
    {
        limiter.RejectionStatusCode = StatusCodes.Status429TooManyRequests; // default is 503
        limiter.AddPolicy(QuoteWrites, http =>
        {
            var limits = http.RequestServices.GetRequiredService<IOptions<RateLimitOptions>>();
            return RateLimitPartition.GetFixedWindowLimiter(
                partitionKey: http.Connection.RemoteIpAddress?.ToString() ?? "unknown",
                factory: _ => new FixedWindowRateLimiterOptions
                {
                    PermitLimit = limits.Value.QuoteWritesPerMinute,
                    Window = TimeSpan.FromMinutes(1),
                    QueueLimit = 0,
                });
        });
    });
}
```

The limit is configuration, validated at start-up like any other option. The policy partitions by client IP with a fixed one-minute window and no queue, and resolves `IOptions<RateLimitOptions>` from the request's services. The test factory sets the limit to 2 to prove the third write gets 429 with a problem+json body.

7 · Calling ~30 rating partners — HttpClientFactory and resilience

`IHttpClientFactory` owns connection pooling and handler lifetimes; a typed client wraps it for one partner API. `AddHttpClient<IPartnerRateClient, PartnerRateClient>` registers the client as transient and gives it an `HttpClient`

whose handler is pooled and recycled every two minutes by default, so DNS changes are picked up (Use the IHttpClientFactory). Store the client in a singleton and that recycling stops working.

Microsoft.Extensions.Http.Resilience, built on Polly, is Resilience4j as one call: `AddStandardResilienceHandler()`. It stacks five strategies in a fixed order with the defaults in table 7.1 and handles 5xx, 408 and 429 responses, `HttpRequestException` and `TimeoutRejectedException` (Build resilient HTTP apps). Its options bind from configuration, so each partner can get its own section instead of its own code.

Two defaults bite an insurance integration. The handler retries *every* HTTP method: a POST that binds cover at a partner can be sent four times, so call `DisableForUnsafeHttpMethods()` — the lab's POST makes one attempt. And the circuit breaker needs at least 100 calls in its 30-second window before it can open, so a low-volume partner never trips it; the lab lowers the minimum to 2 and the circuit opens after two failed attempts. The options are validated too: a sampling duration shorter than twice the attempt timeout is rejected when the client is created.

7.1 · The standard resilience handler, outermost strategy first

#	Strategy	Default	Resilience4j analogue (estimate)	L08.QuoteApi
1	Rate limiter	1,000 concurrent permits, queue 0	Bulkhead	default
2	Total request timeout	30 s, retries included	TimeLimiter around the retry	default
3	Retry	3 retries · exponential · jitter · 2 s base delay	Retry	`MaxRetryAttempts: 3`; tests shorten the delay to 10 ms
4	Circuit breaker	10% failures · min 100 calls · 30 s sampling · 5 s break	CircuitBreaker	`SamplingDuration: 00:00:30`
5	Attempt timeout	10 s per attempt	TimeLimiter per call	`Timeout: 00:00:05`
	Source	● Verified Build resilient HTTP apps		from `appsettings.json`

7.2 · Registering the typed client and its pipeline

C# — samples/L08.QuoteApi/Partners/Partners.cs · #region partner-registration

```
public static IServiceCollection AddPartnerClients(
    this IServiceCollection services, IConfiguration config)
{
    services.AddOptions<PartnerOptions>()
        .Bind(config.GetSection(PartnerOptions.SectionName))
        .ValidateDataAnnotations()
        .ValidateOnStart();
    services.AddSingleton<PartnerHealth>();

    services.AddHttpClient<IPartnerRateClient, PartnerRateClient>((sp, http) =>
        {
            var partners = sp.GetRequiredService<IOptions<PartnerOptions>>().Value;
            http.BaseAddress = new Uri(partners.BaseAddress);
        })
        .AddStandardResilienceHandler(config.GetSection("Partners:Resilience"));
    return services;
}
```

Options feed the client; configuration feeds the resilience pipeline. The base address comes from validated `PartnerOptions`; `AddStandardResilienceHandler` binds `Partners:Resilience`, so retry count and timeouts change per environment without a rebuild. Tests replace only the primary handler, keeping the real pipeline in front of it. `IPartnerRateClient` plays the part of lesson 05 (Async, Tasks & Concurrency)'s `IPartnerRateProvider`, with one difference: a single typed client serves every partner by id, where lesson 05 had one provider per partner.

7.3 · The typed client — retries happen below this code

C# — samples/L08.QuoteApi/Partners/Partners.cs · #region typed-client

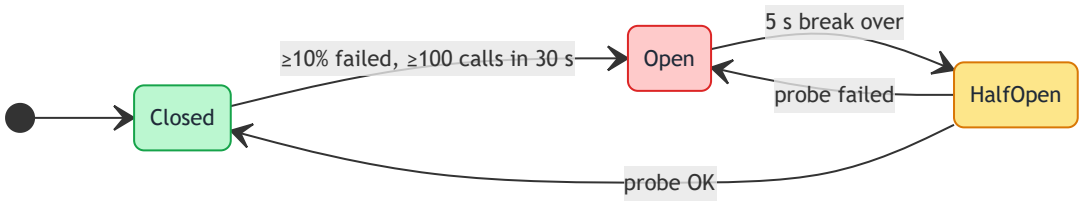
```
/// <summary>Typed client: transient, gets an HttpClient built by IHttpClientFactory.</summary>
public sealed class PartnerRateClient(HttpClient http, PartnerHealth health) : IPartnerRateClient
{
    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web)
    {
        Converters = { new JsonStringEnumConverter() },
```

```
    };

    public async Task<PartnerRate> GetRateAsync(
        string partnerId, CoverageClass coverage, CancellationToken ct)
    {
        try
        {
            var rate = await http.GetFromJsonAsync<PartnerRate>(
                $"rates/{partnerId}?coverage={coverage}", Json, ct);
            health.Record(partnerId, succeeded: true);
            return rate ?? throw new PartnerUnavailableException(partnerId, "empty body");
        }
        catch (Exception ex) when (ex is HttpRequestException or ExecutionRejectedException)
        {
            health.Record(partnerId, succeeded: false);   // retries are already exhausted here
            throw new PartnerUnavailableException(partnerId, ex.Message, ex);
        }
    }
}
```

By the time an exception reaches the `catch`, the pipeline has already retried. `HttpRequestException` means the partner kept failing; `ExecutionRejectedException` covers an open circuit and Polly timeouts. The client records health and throws a domain exception, which 5.3 turns into 503.

7.4 · Circuit breaker states with the standard defaults



green = calls flow · red = calls throw BrokenCircuitException without reaching the partner · amber = one probe call decides · thresholds are the standard handler's defaults

7.5 · Configuring the handler — no retried POST, an early circuit breaker

C# L08.PartnerLab/Program.cs · #region unsafe-methods

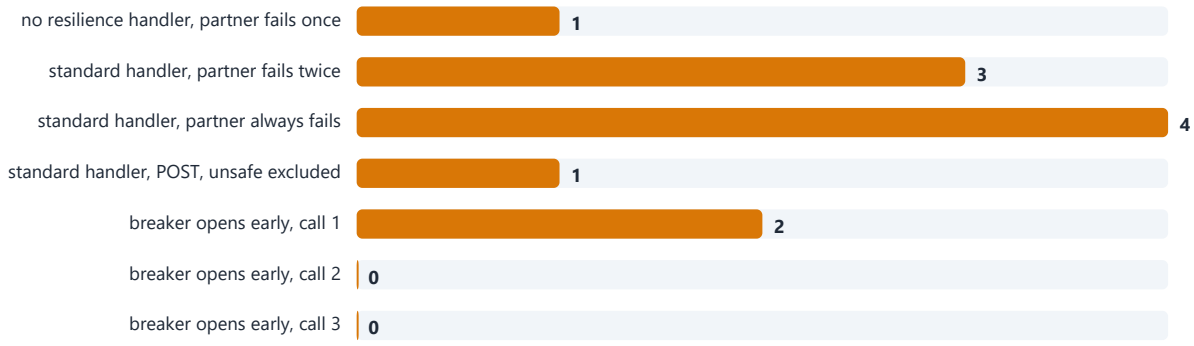
```
builder.AddStandardResilienceHandler(options =>
{
    // default 2 s, exponential: shortened for the lab
    options.Retry.Delay = TimeSpan.FromMilliseconds(10);
    options.Retry.UseJitter = false;
    // POST, PUT, PATCH, DELETE get one attempt only
    options.Retry.DisableForUnsafeHttpMethods();
});
```

C# L08.PartnerLab/Program.cs · #region early-breaker

```
builder.AddStandardResilienceHandler(options =>
{
    options.Retry.Delay = TimeSpan.FromMilliseconds(10);
    options.Retry.UseJitter = false;
    // defaults: 10% failures, at least 100 calls in 30 s
    options.CircuitBreaker.MinimumThroughput = 2;
    options.CircuitBreaker.FailureRatio = 0.5;
});
```

Every strategy in table 7.1 is an options object you can change in code or bind from configuration. Left: `DisableForUnsafeHttpMethods()` keeps retries for GET but gives POST, PUT, PATCH and DELETE a single attempt. Right: `MinimumThroughput = 2` and `FailureRatio = 0.5` let the breaker open after two failed calls instead of waiting for 100 calls in 30 s. Both lab clients shorten `Retry.Delay` and switch off jitter so the lab runs in milliseconds and prints the same counts every time.

7.6 · Partner attempts per scenario in L08.PartnerLab



HTTP attempts that reached the fake partner · outcome in 7.7 · measured from the captured output

Four attempts is the ceiling of a default retry, one is what an unsafe method gets, and zero is an open circuit. The failing GET is tried 1 + 3 times and still ends in 503; the POST with `DisableForUnsafeHttpMethods` is tried once; after two failures the lab's circuit (minimum throughput 2) rejects the next calls without touching the partner (Polly circuit breaker). • **Measured**

7.7 · What the partner lab prints

Output — L08.PartnerLab captured on the build machine

```
scenario                                 attempts  outcome
no resilience handler, partner fails once       1  503
standard handler, partner fails twice           3  200
standard handler, partner always fails          4  503
standard handler, POST, unsafe excluded         1  503
breaker opens early, call 1                     2  BrokenCircuitException
breaker opens early, call 2                     0  BrokenCircuitException
breaker opens early, call 3                     0  BrokenCircuitException
invalid options, rejected when the client is created:
  The sampling duration of circuit breaker strategy needs to be at least double of
  an attempt timeout strategy's timeout interval, in order to be effective.
  Sampling Duration: 30s,Attempt Timeout: 20s
```

The last three lines are a validation failure, not a crash. The lab configures a 20 s attempt timeout with a 30 s sampling duration; the options validator rejects it the first time `CreateClient` builds the pipeline. Validate these values in tests, not in production.

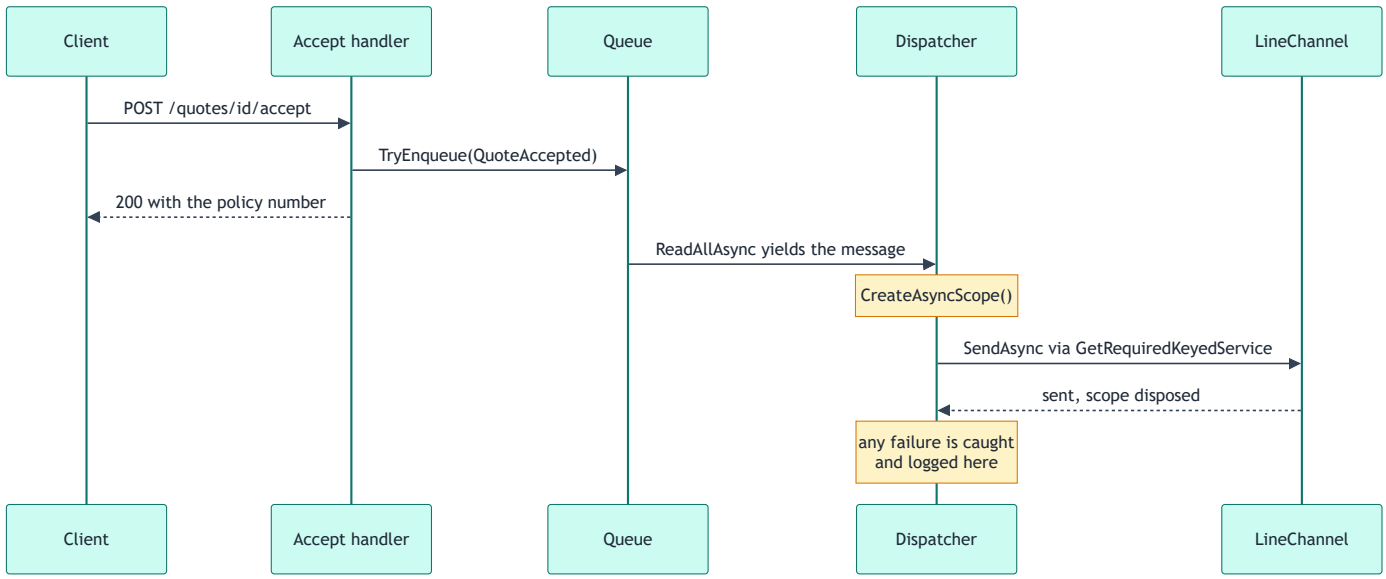
8 · Background work — hosted services

`BackgroundService` is the long-running worker you wrote with `@Scheduled` or a queue consumer, hosted in the API process. Register it with `AddHostedService<T>()`; the host calls `ExecuteAsync` at start-up and cancels its token at shutdown. Since .NET 10 the whole of `ExecuteAsync` runs in the background; before, the code ahead of the first `await` blocked start-up (.NET 10 BackgroundService change).

A hosted service is a singleton, so it creates a scope for each unit of work. The dispatcher reads `QuoteAccepted` messages from a bounded channel, the type you met in lesson 05 (Async, Tasks & Concurrency), opens a scope, resolves the keyed channel the customer chose and sends. Injecting a scoped service into its constructor is the captive dependency of §3.

An exception that escapes `ExecuteAsync` stops the whole host — the default since .NET 6 (.NET 6 hosting exception handling). One bad message would take the API down, so the loop catches per message and logs. An in-memory channel loses work in two ways: when it is full, `TryWrite` returns false and the message is refused, so `QuoteService` checks the result and logs a warning; and whatever is queued is gone when the process stops. For work that must survive a deployment, publish to a durable queue such as SQS — lesson 12 (Cloud-Native .NET on AWS & Modernization).

8.1 · Accepting a quote returns before the notification is sent



Accept handler = request scope · Queue = NotificationQueue singleton · Dispatcher = hosted NotificationDispatcher · LineChannel = keyed, scoped · the client never waits for the send

8.2 · The notification dispatcher

C# — samples/L08.QuoteApi/Notifications/Notifications.cs · #region background-service

```
public sealed class NotificationDispatcher(
    NotificationQueue queue,
    IServiceScopeFactory scopes,
    ILogger<NotificationDispatcher> logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        await foreach (var message in queue.Reader.ReadAllAsync(stoppingToken))
        {
            try   // the whole unit of work: scope, resolution and send
            {
                // a singleton must not hold scoped services: one scope per message
                await using var scope = scopes.CreateAsyncScope();
                var channel = scope.ServiceProvider
                    .GetRequiredKeyedService<INotificationChannel>(message.Channel);
                await channel.SendAsync(message, stoppingToken);
            }
            catch (Exception ex) when (!stoppingToken.IsCancellationRequested)
            {
                // anything escaping ExecuteAsync stops the host: an unknown key, a gateway
                // timeout (TaskCanceledException) and every other failure end up here
                logger.LogError(ex, "Notification {PolicyNumber} via {Channel} failed",
                    message.PolicyNumber, message.Channel);
            }
        }
    }
}
```

The `try` wraps the whole unit of work, and the filter asks the stopping token, not the exception type. `ReadAllAsync(stoppingToken)` ends the loop at shutdown; `CreateAsyncScope()` gives each message fresh scoped services and disposes them asynchronously. Resolving the channel sits inside the `try` because an unknown key throws there. And `when (ex is not OperationCanceledException)` would be wrong: a gateway call that hits `HttpClient.Timeout` throws `TaskCanceledException`, which is an `OperationCanceledException`, so it would escape and stop the host. `DispatcherTests` sends both failures, then a good message, and asserts that it is sent and `/health` still answers.

8.3 · Keyed channels and the hosted service

C# — samples/L08.QuoteApi/Notifications/Notifications.cs · #region keyed-services

```
public static IServiceCollection AddNotifications(this IServiceCollection services)
{
    services.AddSingleton<NotificationQueue>();
    services.AddSingleton<NotificationLog>();
    services.AddKeyedScoped<INotificationChannel, SmsChannel>("sms");
    services.AddKeyedScoped<INotificationChannel, EmailChannel>("email");
    services.AddKeyedScoped<INotificationChannel, LineChannel>("line");
    services.AddHostedService<NotificationDispatcher>();
    return services;
}
```

The key is data from the request. The customer's `notifyVia` value (`sms`, `email` or `line`) travels in the message and selects the keyed registration. The queue and the log are singletons because they are shared between request threads and the dispatcher.

9 · Hands-on — run the API in memory and test it the same way

6 projects, one command to prove them, and a tour that prints what a client sees. `L08.ApiTour` boots `L08.QuoteApi` on the in-memory test server with a fake clock and a fake partner and prints the status codes, bodies and OpenAPI operations a client sees. `L08.HostingLab` and `L08.PartnerLab` are the labs behind 2.5, 3.1, 3.3 and 7.5–7.7.

This lesson's own tests reach the API through `WebApplicationFactory<Program>`, the type that hosts the real `Program` and the real pipeline in memory. That is all §9 uses it for — proving these endpoints, the rate limiter and the resilience handler without a port — and 9.3 shows the two seams it opens. Integration and API testing as a craft (what to double, how much to cover, driving an endpoint test-first) is lesson 10 (Testing, TDD & 100% Coverage).

Visual Basic has no ASP.NET Core project templates, but a VB library can carry web code. Of the 23 web-tagged templates in SDK 10.0.401, 3 offer VB, and all of them are test projects. `L08.Rating.Vb` references the `Microsoft.AspNetCore.App` shared framework, registers its options and maps `/tariff`; the whole VB library, rating rules included, is 183 lines, and the C# host owns `Program.cs`. How far VB goes in a legacy estate is lesson 06 (VB.NET for Legacy Estates).

9.1 · Commands

Terminal

```
# from the repository root, with the .NET 10 SDK first on PATH
dotnet run --project lesson-08-aspnet-core-web-apis/samples/L08.ApiTour
dotnet run --project lesson-08-aspnet-core-web-apis/samples/L08.HostingLab
dotnet run --project lesson-08-aspnet-core-web-apis/samples/L08.PartnerLab
dotnet test lesson-08-aspnet-core-web-apis/samples/L08.QuoteApi.Tests
# build, test and run every lesson-08 sample
python 0-script/verify_samples.py --only 8
# the web templates and the languages they offer
dotnet new list --tag Web
```

No server to start. The tour and the tests host the API in memory, so the commands work in CI without ports or network. `verify_samples.py` builds the web project, runs the tests and runs each console app to completion.

9.2 · What you should see

Output — L08.ApiTour — captured on the build machine

```
POST /quotes                              201  net 8,400.00  total 9,023.95 THB
POST /quotes  (driver 22, 1 claim)        201  net 21,840.00  total 23,462.28 THB
POST /quotes  (3 claims in 5 years)       201  Declined, 3+ claims in 5 years
POST /quotes  (invalid body)              400  errors: Vehicle.SumInsured, NotifyVia
POST /quotes  (5th write this minute)     429  application/problem+json
GET  /quotes/{id}                         200  Quoted
GET  /quotes/{unknown id}                 404  application/problem+json
POST /quotes/{id}/accept                  200  policy MQ-000001, 2026-09-15 to 2027-09-14
POST /quotes/{id}/accept  (again)         409  The quote is AlreadyAccepted.
GET  /tariff/Class3Plus  (VB endpoint)    200  {"coverage":"Class3Plus","baseRate":0.009}
GET  /partners/p07/rates  (fails twice)   200  partner attempts 3
GET  /partners/p09/rates  (always fails)  503  partner attempts 4, Retry-After 30
GET  /health                              200  Degraded
GET  /openapi/v1.json                     200  openapi 3.1.1, 5 paths
```

```
     POST /quotes                          documents 201 400 429
     GET  /quotes/{id}                     documents 200 404
     POST /quotes/{id}/accept              documents 200 404 409
     GET  /tariff/{coverage}               documents 200
     GET  /partners/{partnerId}/rates      documents 200
background notification sent: line  MQ-000001
```

Every line is one real request through the full pipeline. The totals come from the VB calculator on the track's canonical tariff — illustrative rates, not a real one. Both quotes are Class 1 on 800,000 THB (base 2.1% = 16,800): the claim-free 36-year-old with ten licence years takes the 50% no-claim discount to a net 8,400.00 and a total of 9,023.95, and the 22-year-old with one claim takes +20% young driver and +10% claims to 21,840.00 and 23,462.28, with no discount because a claim resets it. Three claims in five years is declined, not priced: this API teaches data, not exceptions, so it stores that outcome as `QuoteStatus.Declined` and still answers 201. The 5th write in the minute gets 429 as problem+json; the partner that fails twice succeeds on the third attempt; the one that always fails ends in 503 with `Retry-After` and turns `/health` Degraded — still 200. The indented lines are the OpenAPI operations counted in 5.1; the last line is written by the background dispatcher.

9.3 · The test factory — the real Program with two seams

C# — samples/L08.QuoteApi.Tests/QuoteApiFactory.cs · #region factory

```csharp
/// <summary>Boots the real Program in memory (TestServer): no port, no network.</summary>
public sealed class QuoteApiFactory : WebApplicationFactory<Program>
{
    public FakeTimeProvider Clock { get; } =
        new(new DateTimeOffset(2026, 9, 1, 9, 0, 0, TimeSpan.Zero));

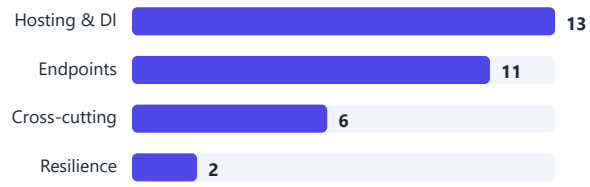
    public FakePartner Partner { get; } = new();

    public int QuoteWritesPerMinute { get; init; } = 1000;

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");      // the factory's default is Development
        builder.UseSetting("Partners:Resilience:Retry:Delay", "00:00:00.010");
        builder.UseSetting("RateLimits:QuoteWritesPerMinute", QuoteWritesPerMinute.ToString());
        builder.UseDefaultServiceProvider(options =>
        {
            options.ValidateScopes = true;      // on by default only in Development
            options.ValidateOnBuild = true;
        });
        builder.ConfigureTestServices(services =>
        {
            services.AddSingleton<TimeProvider>(Clock);
            services.AddHttpClient<IPartnerRateClient, PartnerRateClient>()
                .ConfigurePrimaryHttpMessageHandler(() => Partner);  // no network
        });
    }
}
```

Configuration first, then services. The factory boots `Program.cs` on a `TestServer` — no port, no network — and would default to Development if it named no environment (Integration tests in ASP.NET Core). `UseSetting` overrides a configuration key, which is how the rate-limit test lowers the limit to 2; `ConfigureTestServices` runs after `Program.cs`, so its fake clock and fake partner handler win. Scope validation is on explicitly because the environment is Testing, not Development. This is the seam, not a testing curriculum: lesson 10 (Testing, TDD & 100% Coverage) designs the suite that uses it.

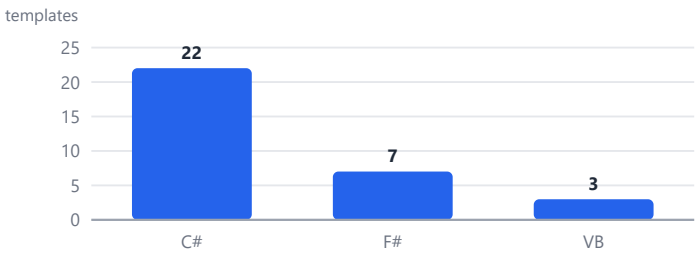
9.4 · Lesson 08 tests by area



[Fact] methods plus [InlineData] rows per test file · measured at build time

32 tests, most of them real HTTP calls through the full pipeline. Endpoint tests share one factory; resilience tests build one each so partner attempt counters start at zero. • Measured

9.5 · Web-tagged SDK templates by language



dotnet new list --tag Web on SDK 10.0.401 · measured from the captured list

C# owns web development; VB appears on 3 templates, all test projects. A VB web API means a C# host plus VB libraries, as here. • Measured

⚠ Traps for a Spring, NestJS or Express engineer

1. **"Scoped beans are checked by the framework."** Only in Development. A singleton that takes a scoped service starts fine in Staging and Production and shares one instance across every request (3.3). Turn `ValidateScopes` and `ValidateOnBuild` on in your test factory.
2. **"Retries are safe to switch on."** The standard resilience handler retries POST, PUT and DELETE too. Against a partner that binds cover or issues a policy, call `DisableForUnsafeHttpMethods()` or make the partner call idempotent (7.5).
3. **"A rejected request is a 429 and a Degraded check is a failure."** The defaults are 503 and 200. Set `RejectionStatusCode`, and map health statuses to what your load balancer and ECS health checks should act on (§6).
4. **"A background job that throws just logs."** An exception escaping `ExecuteAsync` stops the host and every endpoint. Catch per unit of work and let only the stopping token end the loop: an HTTP timeout is a cancellation exception too (8.2).
5. **"The OpenAPI document describes the API."** It describes what was declared. The tour's 503 is missing from the document; add `Produces…` metadata for every code middleware and handlers can return (5.1).

✓ Checkpoint — you are ready for lesson 09 when

1. `python 0-script/verify_samples.py --only 8` reports 6/6 passed, including the 32 API tests.
2. You can read `Program.cs` aloud as diagram 2.1: which middleware runs first, where routing happens and which endpoints opt into rate limiting and caching.
3. You can give every registration in 2.6 and 8.3 its lifetime and explain the two `#3` ids in 3.3.
4. You can explain each status code in 9.2 by naming the component in table 5.2 that produced it.
5. You can configure the standard resilience handler for a partner whose POST must not be retried, and say when its circuit breaker can open.

🔍 Check yourself

1. A service maps endpoints and calls `UseRateLimiter()` but never `UseRouting()`. Does an endpoint's `RequireRateLimiting` policy apply, and why?
2. Which configuration source wins when `appsettings.Production.json`, an environment variable and a command-line argument all set `Quotes:ValidityDays`?
3. In which environment does ASP.NET Core reject a scoped service injected into a singleton, and what happens elsewhere?
4. What must an endpoint return or declare for its 400, 404, 409 and 429 responses to appear in the OpenAPI document?
5. Name the five strategies of the standard resilience handler in order, and the default number of calls before its circuit breaker can open.
6. What status does the rate limiter send by default, and what does `/health` return when a check is Degraded?
7. What happens to the API when an exception escapes `BackgroundService.ExecuteAsync`, and how does the dispatcher prevent it?

Lesson 08 in one line: an ASP.NET Core API is declared, not discovered — explicit lifetimes, middleware in line order behind an implicit `UseRouting`, typed results with .NET 10 validation, an OpenAPI 3.1 document that lists only what you declare, a resilience handler that retries every method, and hosted services that never let an exception escape.
Next: lesson 09 (Data Access with EF Core) — the same quotes and policies in a real database: a `DbContext`, migrations and the SQL EF Core generates, built in projects of its own.